Research Square

Preprints are preliminary reports that have not undergone peer review. They should not be considered conclusive, used to inform clinical practice, or referenced by the media as validated information.

GatherNet: The Lightest Convolutional Neural Network Architecture So far

Wenkang Fan

230202101569 21@stu.xmu.edu.cn

Xiamen University

Xiongbiao Luo

Xiamen University

Research Article

Keywords: Lightweight Networks, Convolutional Neural Networks, Object Detection and Recognition, Depthwise Separable Convolution, ShuttleNet, MobileNet, GhostNet

Posted Date: July 3rd, 2024

DOI: https://doi.org/10.21203/rs.3.rs-4580378/v1

License: This work is licensed under a Creative Commons Attribution 4.0 International License. Read Full License

Additional Declarations: No competing interests reported.

GatherNet: The Lightest Convolutional Neural Network Architecture So far

Wenkang Fan[1] and Xiongbiao Luo[1*]

[1*]Department of Computer Science and Technology, National Institute for Data Science in Health and Medicine, Xiamen University, Xiamen, 361005, China.

*Corresponding author(s). E-mail(s): xiongbiao.luo@gmail.com;
Contributing authors: 23020210156921@stu.xmu.edu.cn;

Abstract

It is generally difficult to establish convolutional neural networks with many operations on mobile devices due to their limited memory and computation resources. This paper proposes a novel Gather module that combines depthwise separable convolution and standard convolution with Ghost module to generate feature maps cheaply and efficiently and uses channel shuffle to rearrange these learned feature maps to improve the information flow between different feature channels. Based on the Gather module, we construct a novel network architecture called GatherNet which is the lightest convolutional neural network architecture so far. We also introduce a hard-swish activation function to effectively solve the data collapse when low-dimensional features are embedded in a high-dimensional space during training. Three benchmark datasets of CIFAR-10, ImageNet-1K, and VOC are used to evaluate our network, with the validation results showing that our proposed GatherNet achieves competitive classification and detection results with much fewer weight parameters than state-of-the-art lightweight network models. Particularly, our GatherNet works much better on a small set of training samples than other lightweight models and still shows the best performance with much fewer parameters and better accuracy when applying it to ocular surface disease recognition. The pre-trained GatherNet model with its code is available at GitHub: https://github.com/Rchen3233/GatherNet.

Keywords: Lightweight Networks, Convolutional Neural Networks, Object Detection and Recognition, Depthwise Separable Convolution, ShuttleNet, MobileNet, GhostNet

Deep convolutional neural networks (CNNs) have shown extraordinary performance in various computer vision tasks [1–3]. However, advanced network models usually require a great number of parameters and computation to achieve good performance balancing between accuracy and speed, which limits the deployment of deep learning models on many mobile and embedded platforms. This issue motivates many researchers to explore portable and efficient network architectures used in edge computing platforms and mobile devices such as smartphones and automatic driving cars [4–8].

Current portable and efficient network models are mainly divided into three categories. On the one hand, many techniques such as network pruning [9, 10], low-rank matrix factorization [11], compact filter [12], and knowledge distillation [13], are used to explore compressed or compact deep convolutional neural networks. Huang et al. [10] quantified the importance of each neuron by calculating the average absolute weight, and pruned the insignificant nodes. Sainath et al. [11] decomposed an original high-dimensional tensor into a low-rank matrix to maintain the potential structure of the compressed data characterization. SqueezeNet [12] employs the fire module to achieve the same performance as AlexNet [14] with less than 50% of the parameters and less than 0.5 MB of the model size. Hinton et al. [13] introduced knowledge distillation to transfer data information from larger models to smaller models. Nevertheless, the performance of these methods is usually confined to the pre-trained model. On the other hand, an efficient neural architecture search (NAS) method has been widely discussed in the literature. MansNet [15] introduces a multi-objective neural network architecture search method based on reinforcement learning, which can find a high-precision and low-time inference CNN architecture. Cai et al. [16] proposed path binarization and demonstrated the close relationship between NAS and model compression. The FBNet family [17, 18] use NAS to build lightweight networks. However, these methods searched in small datasets are not only non-optimal, but require plenty of computing resources to search for the appropriate architecture in large datasets.

More interestingly, many researchers have achieved considerable success in directly designing efficient lightweight architectures with fewer parameters and computation [7, 8, 19–22]. Currently, many works integrate the transformer with convolution to design various hybrid lightweight architectures [23–29]. This work also focuses on creating an efficient lightweight network architecture. As well known, the Ghost module [21] and depthwise separable convolution are efficient and convenient for image feature extraction. Therefore, this work designs a novel Gather module that consists of four key operations of standard convolution, efficient depthwise separable convolution, Ghost module, and channel shuffle to effectively and efficiently learn sufficient image features. Then, we develop a lightest convolutional neural network architecture called GatherNet which uses the Gather module and the dense connection structure that can effectively reuse image features and improve computational efficiency. The technical highlights of this work are summarized as follows.

- A totally novel Gather module is created to parallelly ensemble standard convolution, depthwise separable convolution, and the Ghost module to efficiently and effectively extract image feature maps, which are rearranged by the channel shuffle operation to improve the information flow communication among channels.

- We design a lightest convolutional neural network architecture called GatherNet in accordance with the Gather module and dense connection. Typically, GatherNet uses a hard swish function to alleviate the data folding of fewer output channels. The configuration of GatherNet is detailed in this paper and its implementation code is also publicly available.
- With application to object detection and image classification, GatherNet can critically reduce the number of parameters compared with the current state-of-the-art lightweight networks, especially it achieves the best performance on deeply learned small datasets with small training samples.

The rest of this paper is organized as follows. Section 1 briefly reviews current convolutional lightweight network architectures. Section 1 details the Gather Module and its constructed network architecture. Experimental settings are shown in Section 3. Section compares the experimental results of using different lightweight networks, followed by discussion in Section 5, before concluding this work in Section 5.

1 Current Lightweight Networks

Currently, numerous papers have been published to develop deep convolutional lightweight networks [7, 8, 19, 21, 22] and hybrid lightweight architectures [23–29] in the literature. Specifically, designing lighter and more efficient convolutional neural network architectures with cheaper building units is an effective way to reduce computing resources.

Xception [4] introduces depthwise convolution to more effectively use model parameters. MobileNetV1 [7] proposes depthwise separable convolution to improve the computational efficiency, which was further improved by an inverted residuals module that can reduce the data collapse [19]. ShiftNet [30] reports a shift operation interleaved with point-by-point convolution to significantly improve the depth model efficiency. Zhang et al. [8] employed the group convolution and channel shuffle to build a lightweight CNN architecture ShuffleNetV1, which is recently further updated by four guiding principles [20]. AutoML techniques [31–33] strive for fewer parameters and floating point operations (FLOPs). While Tan et al. [34] developed mixed depthwise convolutional kernels for lightweight networks, Li et al. [35] created MicroNet that integrates factorized depthwise separable convolution into low rank matrix to balance the number of channels and the connectivity of input and output. More recently, GhostNet [21] employs a cheap and efficient Ghost module to generate more diverse image features. CondenseNetV1 [10] adopts learned group convolution and dense connection for feature reuse, further improved by CondenseNetV2 with a sparse feature reactivation unit [22].

On the other hand, lightweight networks are increasingly applied to deal with various computer vision tasks. Kulkarni et al. [36] reported a progressive subtractive recurrent lightweight network for video deraining, Kong et al. [37] introduced a lightweight network model called FastFlowNet for dense optical flow estimation. Yang et al. [38] explored a lightweight network with motion guided attention self-supervised learning for 3D human action recognition. To search for lightweight single image super-resolution models, Huang et al. [39] proposed a differentiable neural architecture

search strategy. Yan et al. [40] built an encoder-decoder structure with a lightweight block to reconstruct multi-frame high dynamic range images with different exposure. More recently, Bao et al. [41] designed a squeeze convolution block with a feature flow enhancement mechanism to address a limited receptive field of lightweight sparse networks, while Zhao et al. [42] established a lightweight architecture of fast blind decontouring networks to deal with synthetic and real-world contouring artifacts.

This work analyzes the advantages and disadvantages of current convolutional lightweight network units, creates an innovative and efficient unit named Gather module, and establishes a new lightweight network architecture called GatherNet with the Gather module that can utilize the advantages of depthwise separable convolution, Ghost, channel shuffle, and dense connection to significantly reduce the model parameters for mobile applications.

2 GatherNet

This section creates a Gather module that integrates depthwise separable convolution, standard convolution, and Ghost operation into channel shuffle for efficient and effective image feature extractions, and constructs a new lightest network architecture GatherNet.

2.1 Gather Module

A lightweight convolutional neural network usually consists of efficient feature extraction filter as a crucial building unit. Among them, many lightweight networks such as MobileNets [7, 19] and GhostNet [21] employ efficient depthwise separable convolution or Ghost module into building blocks. Our novel Gather module employs standard convolution, depthwise separable convolution, and the Ghost module to parallelly extract image features that are effectively communicated between channels using channel shuffle. This module can achieve an excellent trade-off between image feature representation and computational cost (Fig. 1).

Convolution (Conv) takes as the input $\mathbf{X} \in \mathbb{R}^{H \times W \times M}$ and produces a $\mathbb{R}^{H' \times W' \times N}$ feature maps $\mathbf{Y}$ (Fig. 2). The standard convolution is parameterized by convolutional kernel $\mathbf{G}$ of size $K \times K \times M \times N$, and can be written as:

$$\mathbf{Y} = \mathbf{X} \otimes \mathbf{G}, \tag{1}$$

where H, W, H', W' is the spatial height and width of a square input and output feature map, M, N is number of input and output channels, K is the spatial dimension of the kernel, and $\otimes$ denotes the convolution operation. The standard convolution needs the computational cost of:

$$K \cdot K \cdot N \cdot M \cdot H' \cdot W'. \tag{2}$$

Depthwise separable convolution (DWConv) uses a factorized convolution operator instead of the full convolution kernel to decompose the convolution into two independent layers (Fig. 3). The first layer is depthwise convolution, which implements lightweight filter by applying one convolution filter ($K \times K \times 1$) to each input channel.

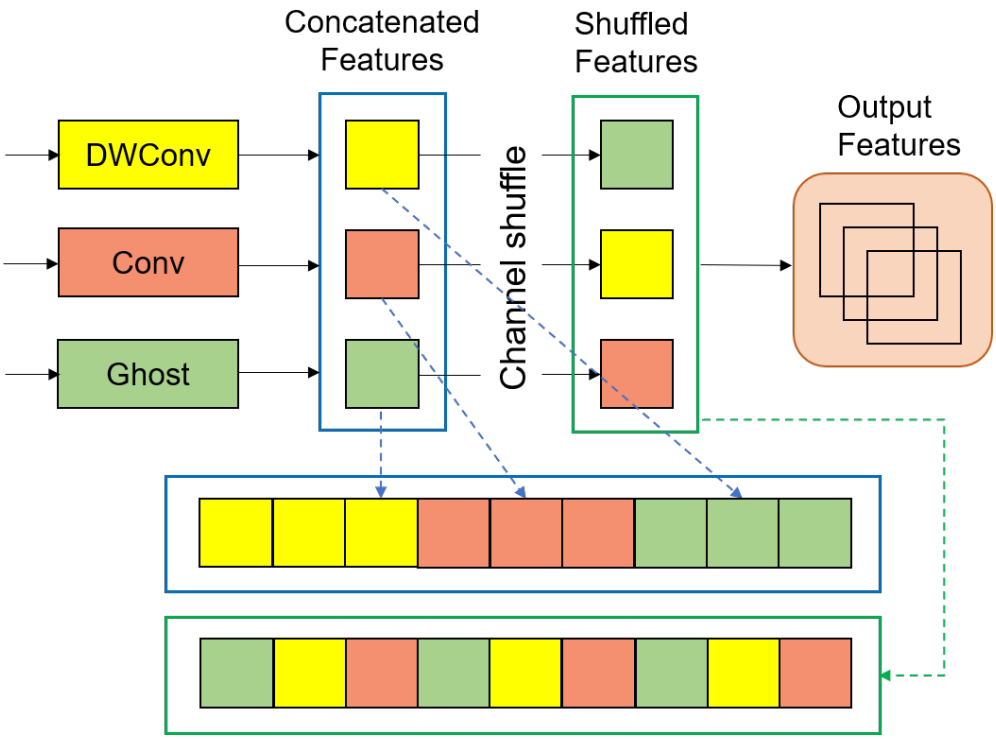


Fig. 1: Gather module is a simple, efficient, and effective module that consists of standard (normal) convolution, depthwise separable convolution, Ghost operator, concatenation, and channel shuffle.

DWConv breaks the interaction between the number of output channels and the size of convolution kernel and produces a new image feature representation $\mathbf{Y}'$:

$$\mathbf{Y}' = \mathbf{X} \otimes \mathbf{G}', \tag{3}$$

where $\mathbf{X} \in \mathbb{R}^{H \times W \times M}$ is the input, $\mathbf{Y}' \in \mathbb{R}^{H' \times W' \times M}$ is the output, and $\mathbf{G}' \in \mathbb{R}^{K \times K \times 1 \times M}$ is the convolution kernel. The computational cost of the depthwise convolution requires is

$$K \cdot K \cdot 1 \cdot M \cdot H' \cdot W'. \tag{4}$$

The second part is pointwise convolution implemented by 1×1 kernel to create a linear combination of depth layer outputs. The pointwise convolution of N output channels can be written as:

$$\mathbf{Y} = \mathbf{Y}' \otimes \mathbf{P}, \tag{5}$$

where $\mathbf{Y}' \in \mathbb{R}^{H' \times W' \times M}$ is the input, $\mathbf{Y} \in \mathbb{R}^{H' \times W' \times N}$ is the output, and $\mathbf{P} \in \mathbb{R}^{1 \times 1 \times M \times N}$ is the convolution kernel. Similarly, the computational cost of the pointwise convolution is

$$1 \cdot 1 \cdot M \cdot N \cdot H' \cdot W'. \tag{6}$$

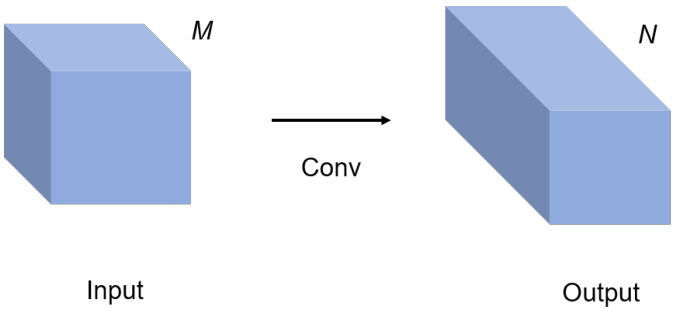


Fig. 2: Convolution operator.

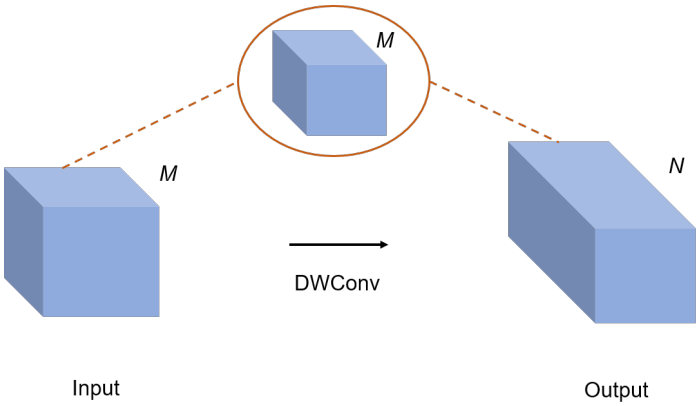


Fig. 3: Depthwise separable convolution.

Consequently, the total computational cost of the depthwise separable convolution is

$$M \cdot H' \cdot W' \cdot (K \cdot K + N), \tag{7}$$

which is less $(1/N + 1/K \cdot K)$ than the standard convolution, e.g., the computational cost of $K = 3$ (3×3 DWConv) is about 9 times smaller than that of 3×3 Conv.

Ghost module is to reduce the required resources, i.e., convolution filters to produce some partial feature maps, and use cheaper linear transformation to generate diverse feature maps for each intermediate feature map (Fig. 4). Specifically, according to Eq. 1, we use the partial-channel ($u << N$) convolution to extract u feature maps:

$$\tilde{\mathbf{Y}} = \mathbf{X} \otimes \tilde{\mathbf{G}} = \{y_{11}, \cdots, y_{ii}, \cdots, y_{uu}\}, i = 1, \cdots, u, \tag{8}$$

where $\tilde{\mathbf{Y}} \in \mathbb{R}^{H' \times W' \times u}$ is the output with u-channel feature maps and $\tilde{\mathbf{G}} \in \mathbb{R}^{K \times K \times M \times u}$ is the convolution kernel. Based on $\tilde{\mathbf{Y}}$, we utilize linear transformation to generate a new feature $\hat{y}_{ij}$:

$$\hat{y}_{ij} = \phi_j(\tilde{y}_{ii}),\ \tilde{y}_i \in \tilde{Y},\ i = 1, \cdots, u,\ j = 1, \cdots, v, \tag{9}$$

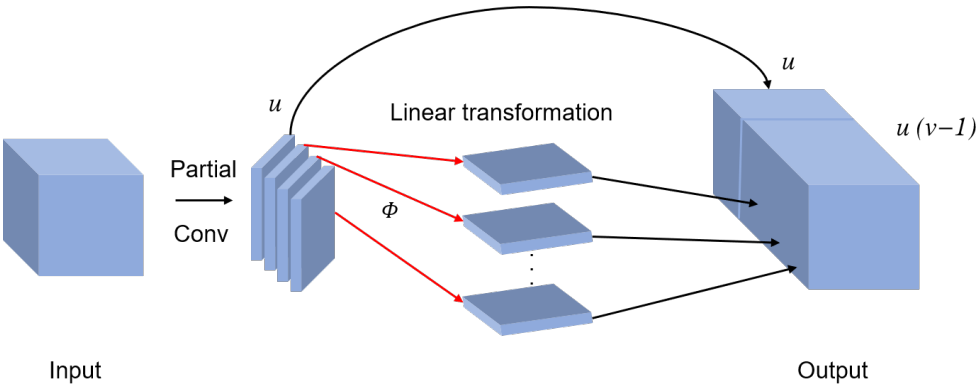


Fig. 4: Ghost module.

where ϕ is the linear transformation that uses the ii-th feature map to generate $(v-1)$ feature maps $(i \neq j)$. So, we obtain $u(v-1)$ new feature maps after the linear transformation. Eventually, $N = u + u(v-1)$ feature maps are obtained by the Ghost module.

Note that the cheap and effective Ghost module can combine with any efficient linear operations such as wavelet transformation [43] and affine transformation [44]. We choose grouping convolution [14] with the similar size of convolution kernel $(D \times D)$ as the linear transformation operation since it shows the better performance than others in our experiments.

The theoretical computational cost speed-up ratio of upgrading standard convolution with the Ghost module is

$$\begin{aligned} r &= \frac{K \cdot K \cdot N \cdot M \cdot H' \cdot W'}{K \cdot K \cdot u \cdot M \cdot H' \cdot W' + D \cdot D \cdot (v-1) \cdot u \cdot H' \cdot W'} \\ &\approx \frac{N \cdot M}{u \cdot M + (v-1) \cdot u} = \frac{v \cdot M}{M + (v-1)} \approx v, \end{aligned} \tag{10}$$

where $D \times D$ has the similar magnitude as that of $K \times K$, and $v << M$.

Channel shuffle is introduced to improve the information communication of feature channels in the Gather module. In tiny networks, a single lightweight unit inclines to meet the complexity constraints by limiting the number of channels which seriously damages the accuracy. Though we connect the outputs of three paths (DWConv, Conv and Ghost) directly stacked together, there will be a side effect: the output of a channel can only be obtained from a small part of the input channel, which critically blocks the information flow between channels [8]. A straightforward solution is to shuffle channels for reorganizing the output feature maps from each path, which can guarantee that the input of later layer comes from different operations. Thus the data information can flow between different channels. Concretely, for the three grouping outputs of DWConv, Conv and Ghost, the total number of channels is $3N$. First, we

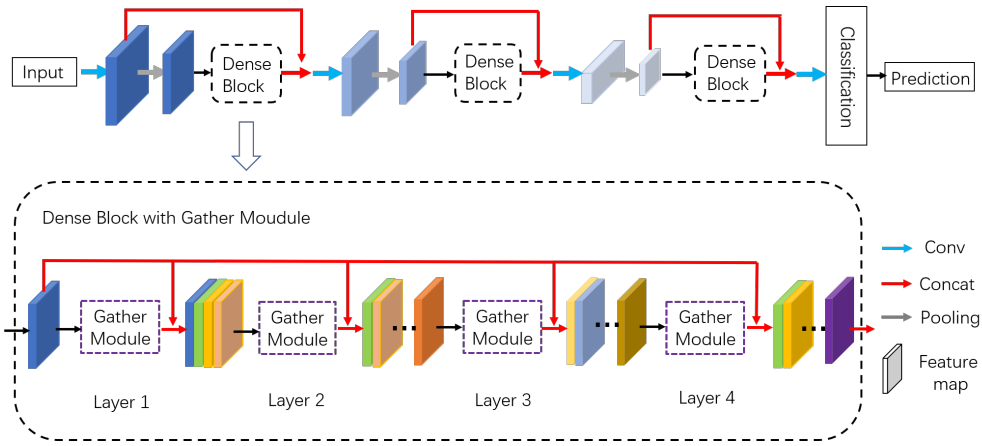


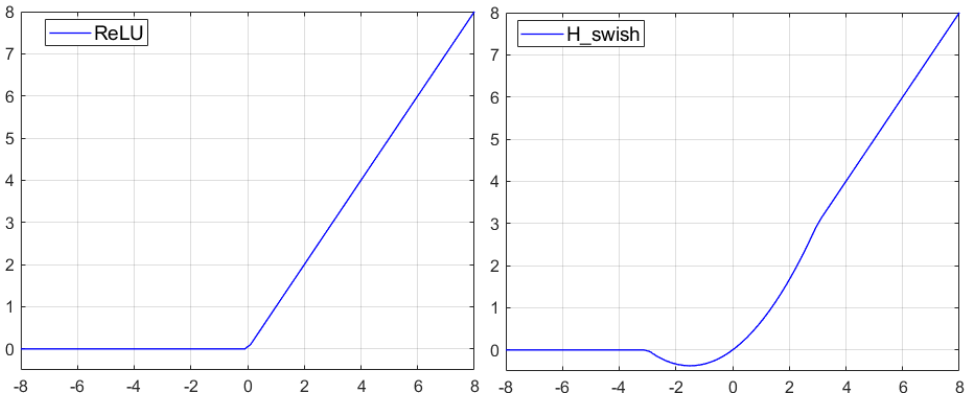
Fig. 5: A GatherNet with three dense blocks. Each dense block consists of four Gather module. The dense blocks between two adjacent blocks are referred to as transition layers and change feature-map sizes via convolution and pooling.

split the channels into two dimensions $(3, N)$, then transpose the two dimensions to get $(N, 3)$, and finally reshape them into one dimension according to the three groups one by one, so that all channels can be rearranged.

2.2 GatherNet

Architecture. It has recently been demonstrated that the long-distance connection in DenseNet architecture will affect the efficiency, and the feature reuse is low in the new learning layer [5]. In this paper, we abandon the concept of enough depth. The architecture of GatherNet follows the principle of full dense connection design. Based on the newly designed Gather module, we develop the lightest network as shown in Fig. 5.

GatherNet consists of an initial Gather module, three dense blocks, two transition blocks and a classification layer. The initial Gather module extracts the global image features. Each dense block includes four Gather modules to extract image features. The output of each path in GatherNet is fixed as N (N represents the growth rate) feature maps, which represent the amount of new information contributed by that path. Each Gather module is followed by a set function of batch normalization (BN), ReLU, and 1×1 Conv. BN can improve the speed, performance and stability of neural network [45]. While ReLU is a non-linearity active function used in training [46]. 1×1 Conv is to reduce the number of channels for improving computational efficiency [47]. A transition block connects two adjacent dense blocks to improve efficiency by reducing the size and number of feature maps. Each transition block consists of four parts: BN, ReLU, DWConv and an average pooling layer. DWConv implemented by 1×1 kernel is to reduce the number of channels by half. The average pooling layer is to average pool with 2×2 kernel [48].



(a) ReLU (b) H_swish

Fig. 6: The non-linearity of two activation functions of ReLU and H_swish

Activation. Considering neural networks with multiple layers, the activation tensor of each layer will form a manifold of interest. When ReLU is used, some points of low-dimensional manifold embedded in high-dimensional space will fold each other, resulting in information loss [19]. This is because the non-zero volume of ReLU has the ability of linear classifier only for the non-zero part. In other words, the use of ReLU for layers with fewer output channels potentially causes data collapse.

In some previous methods [49–51], a nonlinear activation function called swish was introduced. When used as a substitute for ReLU, the non-zero part of swish can significantly improve the expression of features. Since the computional cost of sigmoid function is much higher, we use piece-wise linear H_swish instead of sigmoid function [52]:

$$\mathrm{H-Swish}(\mathrm{x}) = \mathrm{x}\frac{\mathrm{ReLU6}(\mathrm{x}+3)}{6}. \tag{11}$$

Fig. 6 compares the non-linearity of the ReLU and H-swish functions. Our motivation for choosing H_swish is simple. It matches well with the smooth trend of ReLU, and the non-zero volume part different from ReLU can effectively activate features. In addition, H-swish can be implemented as a piece-wise function to significantly reduce the number of memory accesses and the delay cost in practice. At last, we introduce H-swish for those with less than 100 output channels and ReLU for the rest to obtain the relatively consummate manifold of interest.

Table. 1 summarizes the explicit and detailed configuration of GatherNet. Our proposed architecture provides a basic design for reference, although further hyper-parameter tuning or network automatic searching can further boost the performance.

3 Experiments

To verify the effectiveness of the proposed GatherNet architecture, we conduct experiments on benchmark visual datasets for image classification and object detection tasks, including CIFAR-10 [53], ImageNet-1K [55], our in-house ocular surface disease (OSD)

Table 1: Overall architecture of GatherNet. #exp means expansion size. #out means the number of output channels.

Input	Operator	#exp	#out	Stride	Activation
224^2	Gather Module	3	72	2	H_swish
112^2	MaxPool	-	72	2	-
56^2	Gather Module	72	36	1	H_swish
56^2	Gather Module	108	36	1	H_swish
56^2	Gather Module	144	36	1	H_swish
56^2	Gather Module	180	36	1	H_swish
56^2	1×1 DWConv	216	108	1	ReLU
56^2	AvgPool	-	108	2	-
28^2	Gather Module	108	36	1	H_swish
28^2	Gather Module	144	36	1	H_swish
28^2	Gather Module	180	36	1	H_swish
28^2	Gather Module	216	36	1	H_swish
28^2	1×1 DWConv	252	126	1	ReLU
28^2	AvgPool	-	126	2	-
14^2	Gather Module	126	36	1	H_swish
14^2	Gather Module	162	36	1	H_swish
14^2	Gather Module	198	36	1	H_swish
14^2	Gather Module	234	36	1	H_swish
14^2	1×1 Conv	270	270	2	ReLU
7^2	AvgPool	-	270	-	-
1^2	FC	-	1000	-	-

dataset, and PASCAL VOC [56], and compare with state-of-the-art convolutional or transformer-based light-weighted architectures.

Our in-house ocular surface disease dataset was created by three ophthalmologists who used different smartphones with various sizes (e.g., 3648×2736, 4032×3024, 5120×3840) to acquire ocular surface images from 1,247 patients. Three ophthalmologists manually annotated and classified all the acquired ocular surface smartphone images into various OSDs such as conjunctival congestion and pterygium. Eventually, we obtained 1,200 annotated ocular surface smartphone images including 611 normal and 589 abnormal images showing more than 12 different ocular surface diseases. We randomly divided all the annotated data into training and testing by a ratio of $7:3$. The common data preprocessing strategy including random flipping and cropping [1] was applied during training. We resized all the labeled images and found that 512×512 shows the better performance in our experiments.

All the networks are trained using the hyper-parameter growth rate $N = 12$ which indicates the increasing number of feature maps by each layer in the dense block, while the cheap linear transformation in Ghost operation uses $(u, v) = (4, 3)$ and $N = u \cdot v$ to generate new feature maps. We optimize the networks using stochastic gradient descent (SGD) [57] with a momentum of 0.9 [58]. On CIFAR-10, ImageNet-1K, and OSD database we train models using batch size 128, 24, and 16 for 300 epochs, respectively. The initial learning rate is set to 0.15 and is divided by 10 at 50% and 75% of the

Table 2: The top-1 accuracy rates of current lightweight networks on the CIFAR-10 and ImageNet-1K validation sets.

Model	Parameters	FLOPs	Accuracy
Dataset	CIFAR-10 [53]		
MobileNetV3 [54]	1.67M	19.31M	92.87%
GhostNet [21]	3.63M	36.85M	**93.13%**
EdgeViT [26]	3.78M	11.14M	92.58%
EfficientFormerV2 [27]	3.24M	**8.17M**	92.52%
SwiftFormer [28]	3.03M	12.40M	91.30%
GatherNet	**0.24M**	22.12M	90.60%
Dataset	ImageNet-1K [55]		
MobileNetV3 [54]	2.94M	0.89G	73.31%
GhostNet [21]	4.89M	1.67G	73.83%
EdgeViT [26]	4.06M	0.54G	74.61%
EfficientFormerV2 [27]	3.59M	**0.41G**	**75.38%**
SwiftFormer [28]	3.47M	0.61G	74.93%
GatherNet	**0.50M**	1.08G	71.82%

Table 3: Quantitative results of using the lightweight networks evaluated on VOC database.

Backbone	Parameters	FLOPs	mAP
MobileNetV3 [54]	1.92M	3.78G	**50.06%**
GhostNet [21]	3.87M	6.76G	48.03%
EdgeViT [26]	4.02M	2.39G	47.25%
EfficientFormerV2 [27]	3.49M	**1.88G**	46.61%
SwiftFormer [28]	3.27M	2.61G	43.08%
GatherNet	**0.48M**	4.26G	49.17%

total number of training epochs. On the VOC dataset, we use the basic framework of SSD300 [59] and train the models with a 0.001 learning rate for 60k iterations.

4 Results

Several convolutional lightweight network architectures MobileNetV3 [54] and GhostNet [21] or hybrid lightweight networks which integrates CNN and transformer including EdgeViT [26], EfficientFormerV2 [27], and SwiftFormer [28] are compared to our proposed GatherNet.

CIFAR-10 and ImageNet-1K Classification. Table 2 lists the classification accuracy of the different lightweight network architectures. While the accuracy of all the lightweight networks was comparable, our GatherNet can dramatically reduce the number of model parameters from 3.78M and 4.89M to 0.24M and 0.50M. Additionally, convolutional models MobileNetV3 and GhostNet perform better than hybrid models EdgeViT, EfficientFormerV2, and SwiftFormer for CIFAR-10 while hybrid models do better for ImageNet-1K.

VOC Object Detection. To further evaluate the generalization ability of GatherNet, we conduct object detection experiments on the VOC dataset. Here we select SSD300 used as our framework and adopt the backbone feature extractor of replacing

Table 4: Comparison of state-of-the-art efficient deep learning models for ocular surface diseases classification.

Model	Parameters	FLOPs	Accuracy
MobileNetV3 [54]	1.68M	4.63G	86.28%
GhostNet [21]	3.63M	8.72G	86.11%
EdgeViT [26]	3.78M	2.85G	87.37%
EfficientFormerV2 [27]	3.24M	**1.83G**	85.49%
SwiftFormer [28]	3.03M	3.17G	84.38%
GatherNet	**0.24M**	5.66G	**89.72**%
GatherNet w/o Conv	0.11M	2.83G	83.25%
GatherNet w/o DWconv	0.14M	3.53G	85.16%
GatherNet w/o Ghost	0.13M	3.39G	84.39%
GatherNet w/o Channel Shuffle	0.24M	5.66G	87.29%
GatherNet w/o Swish Activation	0.24M	5.66G	87.52%

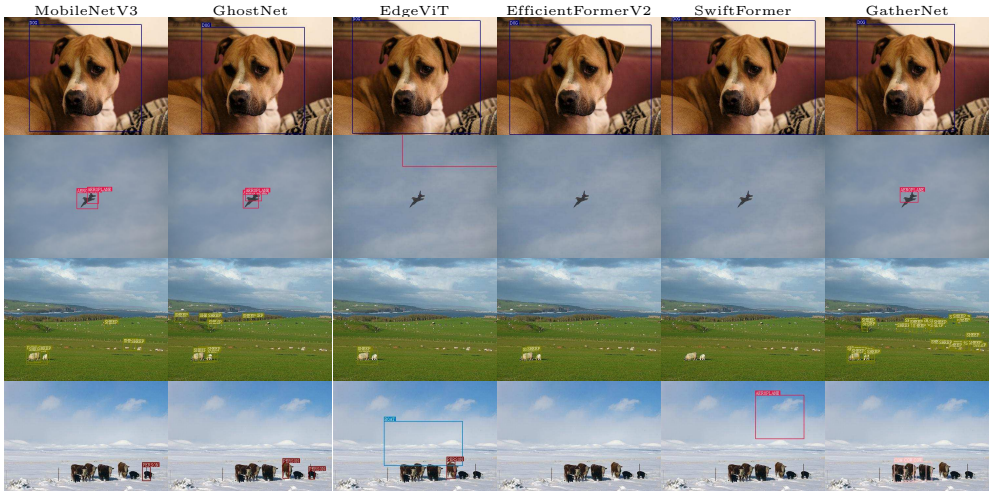


Fig. 7: Visual comparison of object detection results of using lightweight models of MobileNetV3 [54], GhostNet [21], EdgeViT [26], EfficientFormerV2 [27], SwiftFormer [28], and GatherNet (Ours) evaluated on VOC database

original VGG-16 [60] with lightweight models. Fig. 7 and Table 3 show the object detection results. Most remarkably, MobileNetV3, GhostNet, and our GatherNet perform better than EdgeViT, EfficientFormerV2, and SwiftFormer in small object detection. With significantly lower parameters and an acceptable amount of computation, GatherNet achieves a similar mAP to MobileNetV3 and performs better than others.

OSD Classification. One of our motivations is to explore lightweight models embedded in smartphones to recognize OSD diseases in smartphone camera images, which can let patients diagnose themselves without going to see a doctor. Fig. 8 shows some examples of OSDs recognition using our GatherNet. Table 4 illustrates the results of using different lightweight architectures for automatic OSD recognition and an ablation study to demonstrate the effectiveness of various components of GatherNet.

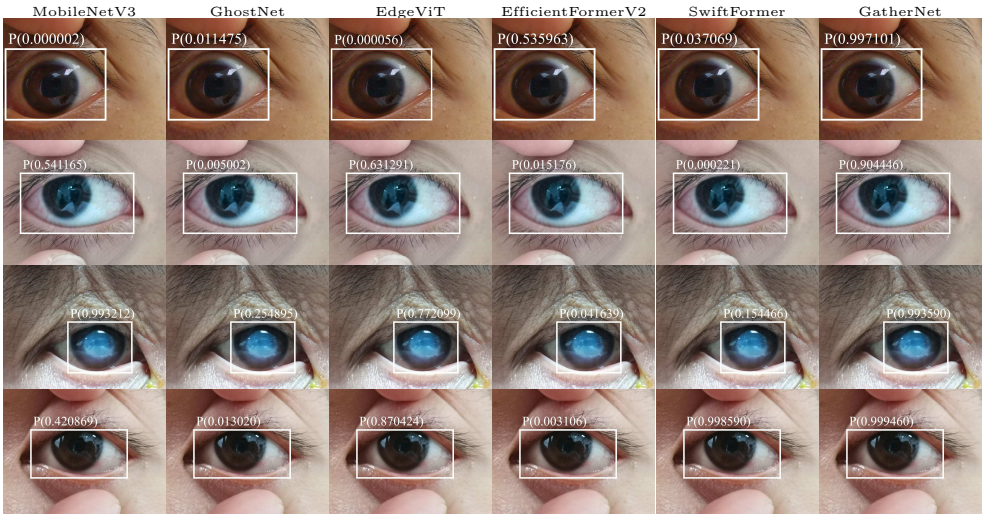


Fig. 8: Visual comparison of the different lightweight networks for OSDs recognition. **P**(∗) represents the probability of classification correctly. The classification results in Columns 1∼6 correspond to the methods of MobileNetV3 [54], GhostNet [21], EdgeViT [26], EfficientFormerV2 [27], SwiftFormer [28], and GatherNet, respectively.

We can see that GatherNet only has 0.24M weights and provides an average accuracy of 89.72%. It can be observed that GatherNet with a higher accuracy rate and fewer parameters obviously outperforms all other approaches.

5 Discussion

One technical motivation is to address the over-fitting problem when using small data to train lightweight networks that usually learn too many redundant features during the training process. Another motivation is to construct simpler network architectures that are in demand for limited computational resource platforms for object recognition (detection and classification) in mobile devices. Therefore, we propose GatherNet to learn diverse features through the Gather modules and avoid the interference of redundant features on the model performance, improving the generalization ability. The effectiveness and limitations of GatherNet are discussed as follows.

5.1 Effectiveness

Compared to other lightweight network models generating too many abundant feature maps in each convolution or transformer layer, GatherNet is a more ingenious network architecture with several Gather modules, working well on small training datasets and hardly getting trapped in over-fitting, as well as without generating many abundant features. This success is certainly attributed to the structure of the Gather module that integrates standard convolution, depthwise separable convolution, and the Ghost module into the channel shuttle. Specifically, the input of each Gather module comes

from the concatenation of relatively few feature maps generated by the previous Gather module, which can further effectively use these image features and avoid the interference of too many redundant features. Next, GatherNet benefits from dense connections that can effectively reuse image feature maps, significantly reducing the number of parameters. Additionally, we introduce a non-linearity H-swish activation function into GatherNet, enabling it to more precise feature representation. More interestingly, we can see the convolutional lightweight model MobileNetV3 [54] and GhostNet [21] perform better than hybrid models EdgeViT [26], EfficientFormerV2 [27], and SwiftFormer [28] in CIFAR-10, VOC, and our OSD dataset since local detail features are more important for these tasks, while hybrid models perform better for ImageNet-1K since the transformer extracts many global features better for high-resolution image classification.

5.2 Limitations

Although GatherNet works efficiently and effectively for image classification and detection, it still suffers from several limitations. On the one hand, GatherNet is the lightest network structure with limited parameters but it is generally unable to extract sufficient features from large training data. This explains why the accuracy of GatherNet was slightly worse than other lightweight network models (Tables 2). On the other hand, the computational efficiency of GatherNet is not as good as hybrid lightweight models while our model has less than one-tenth the parameters of other models.

6 Conclusion

This work designs a new cheap building unit called Gather module that integrates standard convolution, depthwise separable convolution, Ghost module into the channel shuffle operation to generate internal feature maps and improve their information flow between channels. We combine the Gather module and dense connection with the a hard swish activation function to create a new lightest network architecture called GatherNet so far. With application to image classification and detection, the experimental results demonstrate that GatherNet has comparable results with fewer parameters.

References

[1] He, K., Zhang, X., Ren, S., Sun, J.: Deep residual learning for image recognition. In: CVPR (2016)

[2] Redmon, J., Divvala, S., Girshick, R., Farhadi, A.: You only look once: Unified, real-time object detection. In: CVPR, pp. 779–788 (2016)

[3] Ronneberger, O., Fischer, P., Brox, T.: U-net: Convolutional networks for biomedical image segmentation. In: MICCAI, pp. 234–241 (2015)

[4] Chollet, F.: Xception: Deep learning with depthwise separable convolutions. In: CVPR (2017)

[5] Huang, G., Liu, Z., Laurens, V., Weinberger, K.Q.: Densely connected convolutional networks. IEEE Computer Society (2016)

[6] Tan, M., Le, Q.: EfficientNet: Rethinking model scaling for convolutional neural networks. In: ICML, pp. 6105–6114 (2019)

[7] Howard, A.G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M., Adam, H.: Mobilenets: Efficient convolutional neural networks for mobile vision applications. arXiv preprint arXiv:1704.04861 (2017)

[8] Zhang, X., Zhou, X., Lin, M., Sun, J.: Shufflenet: An extremely efficient convolutional neural network for mobile devices. In: CVPR, pp. 6848–6856 (2018)

[9] Luo, J.-H., Wu, J., Lin, W.: Thinet: A filter level pruning method for deep neural network compression. In: ICCV, pp. 5058–5066 (2017)

[10] Huang, G., Liu, S., Maaten, L., Weinberger, K.Q.: Condensenet: An efficient densenet using learned group convolutions. In: CVPR, pp. 2752–2761 (2018)

[11] Sainath, T.N., Kingsbury, B., Sindhwani, V., Arisoy, E., Ramabhadran, B.: Low-rank matrix factorization for deep neural network training with high-dimensional output targets. In: 2013 IEEE International Conference on Acoustics, Speech and Signal Processing, pp. 6655–6659 (2013). IEEE

[12] Iandola, F.N., Han, S., Moskewicz, M.W., Ashraf, K., Dally, W.J., Keutzer, K.: Squeezenet: Alexnet-level accuracy with 50x fewer parameters and¡ 0.5 mb model size. arXiv preprint arXiv:1602.07360 (2016)

[13] Hinton, G., Vinyals, O., Dean, J.: Distilling the knowledge in a neural network. Computer Science **14**(7), 38–39 (2015)

[14] Krizhevsky, A., Sutskever, I., Hinton, G.E.: Imagenet classification with deep convolutional neural networks. Advances in neural information processing systems **25**, 1097–1105 (2012)

[15] Tan, M., Chen, B., Pang, R., Vasudevan, V., Sandler, M., Howard, A., Le, Q.V.: Mnasnet: Platform-aware neural architecture search for mobile. In: CVPR, pp. 2820–2828 (2019)

[16] Cai, H., Zhu, L., Han, S.: Proxylessnas: Direct neural architecture search on target task and hardware. arXiv preprint arXiv:1812.00332 (2018)

[17] Wu, B., Dai, X., Zhang, P., Wang, Y., Sun, F., Wu, Y., Tian, Y., Vajda, P., Jia, Y., Keutzer, K.: Fbnet: Hardware-aware efficient convnet design via differentiable neural architecture search. In: CVPR, pp. 10734–10742 (2019)

[18] Wan, A., Dai, X., Zhang, P., He, Z., Tian, Y., Xie, S., Wu, B., Yu, M., Xu, T., Chen, K., *et al.*: Fbnetv2: Differentiable neural architecture search for spatial and

channel dimensions. In: CVPR, pp. 12965–12974 (2020)

[19] Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., Chen, L.-C.: Mobilenetv2: Inverted residuals and linear bottlenecks. In: CVPR, pp. 4510–4520 (2018)

[20] Ma, N., Zhang, X., Zheng, H.-T., Sun, J.: Shufflenet v2: Practical guidelines for efficient cnn architecture design. In: ECCV, pp. 116–131 (2018)

[21] Han, K., Wang, Y., Tian, Q., Guo, J., Xu, C., Xu, C.: Ghostnet: More features from cheap operations. In: CVPR, pp. 1580–1589 (2020)

[22] Yang, L., Jiang, H., Cai, R., Wang, Y., Song, S., Huang, G., Tian, Q.: Condensenet v2: Sparse feature reactivation for deep networks. In: CVPR, pp. 3569–3578 (2021)

[23] Mehta, S., Rastegari, M.: Mobilevit: light-weight, general-purpose, and mobile-friendly vision transformer. arXiv preprint arXiv:2110.02178 (2021)

[24] Mehta, S., Rastegari, M.: Separable self-attention for mobile vision transformers. arXiv preprint arXiv:2206.02680 (2022)

[25] Maaz, M., Shaker, A., Cholakkal, H., Khan, S., Zamir, S.W., Anwer, R.M., Shahbaz Khan, F.: Edgenext: efficiently amalgamated cnn-transformer architecture for mobile vision applications. In: European Conference on Computer Vision, pp. 3–20 (2022). Springer

[26] Pan, J., Bulat, A., Tan, F., Zhu, X., Dudziak, L., Li, H., Tzimiropoulos, G., Martinez, B.: Edgevits: Competing light-weight cnns on mobile devices with vision transformers. In: European Conference on Computer Vision, pp. 294–311 (2022). Springer

[27] Li, Y., Hu, J., Wen, Y., Evangelidis, G., Salahi, K., Wang, Y., Tulyakov, S., Ren, J.: Rethinking vision transformers for mobilenet size and speed. In: Proceedings of the IEEE/CVF International Conference on Computer Vision, pp. 16889–16900 (2023)

[28] Shaker, A., Maaz, M., Rasheed, H., Khan, S., Yang, M.-H., Khan, F.S.: Swiftformer: Efficient additive attention for transformer-based real-time mobile vision applications. In: Proceedings of the IEEE/CVF International Conference on Computer Vision, pp. 17425–17436 (2023)

[29] Xie, X., Wu, D., Xie, M., Li, Z.: Ghostformer: Efficiently amalgamated cnn-transformer architecture for object detection. Pattern Recognition **148**, 110172 (2024)

[30] Wu, B., Wan, A., Yue, X., Jin, P., Zhao, S., Golmant, N., Gholaminejad, A., Gonzalez, J., Keutzer, K.: Shift: A zero flop, zero parameter alternative to spatial

convolutions. In: CVPR (2018)

[31] Zoph, B., Vasudevan, V., Shlens, J., Le, Q.V.: Learning transferable architectures for scalable image recognition. In: CVPR (2018)

[32] Gong, X., Chang, S., Jiang, Y., Wang, Z.: Autogan: Neural architecture search for generative adversarial networks. In: ICCV (2019)

[33] Yang, Z., Wang, Y., Chen, X., Shi, B., Xu, C., Xu, C., Tian, Q., Xu, C.: Cars: Continuous evolution for efficient neural architecture search. In: CVPR (2020)

[34] Tan, M., Le, Q.V.: Mixconv: Mixed depthwise convolutional kernels. CoRR **abs/1907.09595** (2019)

[35] Li, Y., Chen, Y., Dai, X., Chen, D., Liu, M., Yuan, L., Liu, Z., Zhang, L., Vasconcelos, N.: Micronet: Towards image recognition with extremely low flops. CoRR **abs/2011.12289** (2020)

[36] Kulkarni, A., Patil, P.W., Murala, S.: Progressive subtractive recurrent lightweight network for video deraining. IEEE Signal Processing Letters **29**, 229–233 (2021)

[37] Kong, L., Shen, C., Yang, J.: FastFlowNet: A Lightweight Network for Fast Optical Flow Estimation. In: IEEE International Conference on Robotics and Automation (ICRA), p. 21459746 (2021)

[38] Yang, Y., Liu, G., Gao, X.: Motion guided attention learning for self-supervised 3d human action recognition. IEEE Transactions on Circuits and Systems for Video Technology **32**, 8623–8634 (2022)

[39] Huang, H., Shen, L., He, C., Dong, W., Liu, W.: Differentiable neural architecture search for extremely lightweight image super-resolution. IEEE Transactions on Circuits and Systems for Video Technology, (2022)

[40] Yan, Q., Zhang, S., Chen, W., Liu, Y., Zhang, Z., Zhang, Y., Shi, J.Q., Gong, D.: A Lightweight Network for High Dynamic Range Imaging. In: IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops, pp. 824–832 (2022)

[41] Bao, Z., Yang, S., Huang, Z., Zhou, M., Chen, Y.: A lightweight block with information flow enhancement for convolutional neural networks. IEEE Transactions on Circuits and Systems for Video Technology, (2023)

[42] Zhao, Y., Jia, W., Chen, Y., Wang, R.: Fast blind decontouring network. IEEE Transactions on Circuits and Systems for Video Technology **33**, 478–490 (2023)

[43] Chen, C.-H., Lee, J.-S., Sun, Y.-N.: Wavelet transformation for gray-level corner detection. Pattern Recognition **28**(6), 853–861 (1995)

[44] Dong, P., Galatsanos, N.P.: Affine transformation resistant watermarking based on image normalization. In: Proceedings. International Conference on Image Processing, vol. 3, pp. 489–4923 (2002)

[45] Ioffe, S., Szegedy, C.: Batch normalization: Accelerating deep network training by reducing internal covariate shift. In: ICML, vol. 37, pp. 448–456 (2015)

[46] Glorot, X., Bordes, A., Bengio, Y.: Deep sparse rectifier neural networks. In: Gordon, G., Dunson, D., Dud?k, M. (eds.) Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics, vol. 15, pp. 315–323 (2011)

[47] Szegedy, C., Vanhoucke, V., Ioffe, S., Shlens, J., Wojna, Z.: Rethinking the inception architecture for computer vision. In: CVPR (2016)

[48] Lecun, Y., Bottou, L., Bengio, Y., Haffner, P.: Gradient-based learning applied to document recognition. Proceedings of the IEEE **86**(11), 2278–2324 (1998)

[49] Elfwing, S., Uchibe, E., Doya, K.: Sigmoid-weighted linear units for neural network function approximation in reinforcement learning. Neural Networks **107**, 3–11 (2018)

[50] Hendrycks, D., Gimpel, K.: Bridging nonlinearities and stochastic regularizers with gaussian error linear units. CoRR **abs/1606.08415** (2016)

[51] Prajit Ramachandran, B.Z., Le, Q.V.: Searching for activation functions. CoRR **abs/1710.05941** (2017)

[52] Avenash, R., Vishawanth, P.: Semantic segmentation of satellite images using a modified cnn with hard-swish activation function. In: VISIGRAPP (2019)

[53] Krizhevsky, A., Hinton, G.: Learning multiple layers of features from tiny images. Handbook of Systemic Autoimmune Diseases **1**(4) (2009)

[54] Howard, A., Sandler, M., Chu, G., Chen, L.-C., Chen, B., Tan, M., Wang, W., Zhu, Y., Pang, R., Vasudevan, V., *et al.*: Searching for mobilenetv3. In: Proceedings of the IEEE/CVF International Conference on Computer Vision, pp. 1314–1324 (2019)

[55] Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., Fei-Fei, L.: Imagenet: A large-scale hierarchical image database. In: 2009 IEEE Conference on Computer Vision and Pattern Recognition, pp. 248–255 (2009). Ieee

[56] Everingham, M., Van Gool, L., Williams, C.K., Winn, J., Zisserman, A.: The pascal visual object classes (voc) challenge. International journal of computer vision **88**(2), 303–338 (2010)

[57] Bottou, L.: Large-scale machine learning with stochastic gradient descent.

Physica-Verlag HD (2010)

[58] Sutskever, I., Martens, J., Dahl, G., Hinton, G.: On the importance of initialization and momentum in deep learning. In: ICML, vol. 28, pp. 1139–1147 (2013)

[59] Liu, W., Anguelov, D., Erhan, D., Szegedy, C., Reed, S., Fu, C.-Y., Berg, A.C.: Ssd: Single shot multibox detector. In: European Conference on Computer Vision, pp. 21–37 (2016). Springer

[60] Simonyan, K., Zisserman, A.: Very deep convolutional networks for large-scale image recognition. arXiv preprint arXiv:1409.1556 (2014)